Policy Registry sandbox review

Source reviewed: `692022e1ca34a280f90a198d2455ac3535b3bce2`. Evidence was rerun on 9 November 2026 against branch base `2fbcc41882a3c84dd7efc8defa6768ef0bccf4d8`. This is a focused source and sandbox review. It is not a security audit.
The package contains no sukuk token. Its plain dummy token is GUSD. On a fresh local chain, both `transfer` and `transferFrom` succeeded after GUSD was unapproved; direct `transfer` also succeeded while the registry was paused.

Glossary

- **Whitelist:** owner-maintained list of permitted addresses.
- **Executor:** a calling contract allowed to request a guarded registry check.
- **Revert:** a contract call fails and rolls back its changes.
- **Pause:** a registry state in which guarded checks revert.
- **Policy version:** a counter advanced when a registry list changes.
- **Hash:** fixed-size fingerprint; each policy-change hash includes the preceding hash.
- **Snapshot:** a local-chain save point that can restore test state.

Three answers

What is permitted? `checkSubscriptionCompliance` checks the asset address and `msg.sender` executor status, then returns the current policy version. It ignores `subscriber`; it does not judge investor eligibility, amount or transfer parties.
Can the check be bypassed? Yes. The current plain GUSD token's `transfer` and `transferFrom` do not call the registry. The local transfer succeeded even when the registry rejected the asset. The migration contracts have guarded asset checks; this is not enforcement for every token route.
What happens during an outage? Paused on-chain checks revert and the guarded migration paths stop. The file-driven HTTP mock is not registry-backed. There is no production HTTP transfer client or specified timeout, retry or stale-decision rule.
Review branch files and evidence.

Local sandbox access

Use Bash, Git, Python 3, Node.js 22 with npm and Foundry 1.9.0. Check `forge --version`, `cast --version` and `anvil --version`. Foundry installation instructions: getfoundry.sh/introduction/installation.
Clone the review branch, then build and test:

```
git clone --branch mb-sandbox-review --recurse-submodules \
  https://github.com/AbZe628/gravitas-protocol.git
cd gravitas-protocol
git submodule update --init --recursive
forge build
forge test
```

Expected: build succeeds; 86 tests pass. Start a private chain in terminal 1. Never expose it or use these accounts with real funds.

```
anvil --host 127.0.0.1 --port 8545 --chain-id 31337 --mnemonic \
  'test test test test test test test test test test test junk'
```

From the repository root in terminal 2:

```
bash mb-sandbox-review/scripts/setup-local.sh
source /tmp/mb-sandbox-env.sh
python3 mb-sandbox-review/scripts/check-local.py
```

Expected: version 3, six `PASS` lines and restored version-3 baseline. The dummy token is GUSD, not a sukuk. Full instructions, API details and reproducible outputs are in the review branch.

01 — Simulated tokenised sukuk transfer

This scenario uses plain GUSD as a stand-in; no sukuk token exists here. It answers three questions: the registry permits only an approved asset and executor on its guarded check; `transfer` and `transferFrom` bypass that check in the present token; and a failed guarded registry call blocks a migration, while HTTP outage handling is not implemented. Follow sandbox-access.md to deploy the local chain and set the environment first. It is not a deployment script.

With a fresh version-3 baseline, run the read calls:

```
cast call "$REGISTRY" 'checkSubscriptionCompliance(address,address)(uint256)' "$RECIPIENT" \
  "$ASSET" --from "$EXECUTOR" --rpc-url "$RPC"
cast call "$REGISTRY" 'checkSubscriptionCompliance(address,address)(uint256)' "$RECIPIENT" \
  "$UNKNOWN" --from "$EXECUTOR" --rpc-url "$RPC"
cast call "$REGISTRY" 'checkSubscriptionCompliance(address,address)(uint256)' "$RECIPIENT" \
  "$ASSET" --from "$UNKNOWN" --rpc-url "$RPC"
cast call "$REGISTRY" 'checkSubscriptionCompliance(address,address)(uint256)' "$UNKNOWN" \
  "$ASSET" --from "$EXECUTOR" --rpc-url "$RPC"
```

Expected: first call returns 3; unapproved asset and caller revert; the last call also returns 3 because the subscriber is ignored.

The actual direct and delegated transfer bypasses, paused-registry checks and balances are tested by:

```
python3 mb-sandbox-review/scripts/check-local.py
forge test --match-path test/GravitasSubscriptionIntegration.t.sol -vv
forge test --match-test \
  'test_Registry_(VerifyFunctionsRevertWhenPaused|PauseHaltsExecutorAuthorizationSystemWide|
OwnerNotLockedOutOfAuthGateByRegistryPause)' \
  -vv
forge test --match-path test/foundry/TeleportFlow.t.sol --match-test '^test_V2_RescueTokens' \
  -vv
```

Expected: the local helper prints six `PASS` lines; it successfully transfers GUSD after revocation and during pause, and restores version 3. The named Foundry commands select existing tests. Guarded migration checks fail closed on pause; the direct token route never checks the registry. HTTP outage behavior for a production transfer client is not specified or tested.

02 — Permitted pair

The integration fixture whitelists dummy asset A at version 2 and a dummy executor at version 3. These checks read address flags; they do not move tokens. Run from the repository root:

```
forge test --match-path test/GravitasSubscriptionIntegration.t.sol --match-test \
  '^test_VerifyAssetCompliance' -vv
forge test --match-path test/GravitasSubscriptionIntegration.t.sol --match-test \
  '^test_AreTokensCompliant' -vv
```

Expected: A returns `true`; unlisted address `0x000000000000000000000000000000000000dead` returns `false`. After the fixture adds asset B (`...0101`), `areTokensCompliant(A, B)` returns `true`; replacing B with an unlisted address returns `false`. The proof is in `test_VerifyAssetCompliance` and `test_AreTokensCompliant` in `test/GravitasSubscriptionIntegration.t.sol`. Neither check evaluates investor, amount, transfer parties or caller authorization.

03 — Prohibited asset or caller

The fresh integration fixture approves one dummy asset and one executor. The combined check rejects an unapproved asset and rejects a request from an unapproved caller. Run from the repository root:

```
forge test --match-path test/GravitasSubscriptionIntegration.t.sol --match-test \
  '^test_CheckSubscriptionCompliance_RevertOnNonCompliantAsset' -vv
forge test --match-path test/GravitasSubscriptionIntegration.t.sol --match-test \
  '^test_CheckSubscriptionCompliance_RevertOnUnauthorizedExecutor' -vv
forge test --match-path test/GravitasSubscriptionIntegration.t.sol --match-test \
  '^test_Verify(AssetCompliance|ExecutorStatus)' -vv
```

Expected: first case reverts with `GPR: Asset not Shariah-compliant`; second with `GPR: Calling contract not an authorized executor`. Boolean verify functions return `false` for an unapproved entry while unpaused; they do not themselves stop a transfer. The named test functions in `test/GravitasSubscriptionIntegration.t.sol` prove each result. The subscriber parameter is not checked.

04 — Policy version and change hash

The registry starts at version 1 after granting executor status to its deployer. A real change advances the version and records a hash that includes the previous hash; a duplicate write changes neither. Pause and unpause do not update the policy lists. Run from the repository root:

```
forge test --match-contract '^PolicyCommitment' -vv
forge test --match-contract '^GravitasSubscriptionIntegrationTest' --match-test \
  '^test_CheckSubscriptionCompliance_Success' -vv
forge test --match-contract '^PausableTests' --match-test '^test_Registry_PauseUnpause' -vv
```

Expected: all selected tests pass. `test_EachVersionFoldsInTheOneBeforeIt`, `test_RewritingTheSameValueIsNotAPolicyChange` and `test_OrderOfChangesIsPartOfTheCommitment` in `test/foundry/PolicyCommitment.t.sol` check the sequence. `policyHistory` stores change hashes, not the full historical registry state or a board ruling. A replay needs the ordered changes.

05 — Paused registry and unavailable HTTP service

On a fresh version-3 local chain, the guarded registry checks fail while paused; direct GUSD transfer still succeeds because the token never calls the registry. The HTTP mock is a separate file-driven service. Its production outage policy is not implemented. Run the local probes and existing tests from the repository root:

```
python3 mb-sandbox-review/scripts/check-local.py
forge test --match-path test/foundry/PausableTests.t.sol --match-contract '^PausableTests' \
  --match-test '^test_Registry_' -vv
```

Expected: six local `PASS` lines; the helper tests all five guarded calls, raw getters, direct transfer and restores the snapshot. Foundry selects six existing pause tests, including `test_Registry_VerifyFunctionsRevertWhenPaused` and `test_Registry_PauseHaltsExecutorAuthorizationSystemWide`.

With the mock stopped, this request should fail at the transport layer:

```
curl --max-time 2 http://127.0.0.1:8787/v1/policies
```

The error is not an `ALLOW` decision and does not prove production transfer handling. Timeout, retries and stale-decision rules remain to be specified and tested.

06 — Unauthorised policy change

Only the registry owner can change asset, router or executor lists. `Ownable2Step` ownership transfer takes effect only after the proposed owner accepts. The local probe also checks that a pending owner cannot write before acceptance and that the wrong account cannot accept. Run from the repository root:

```
python3 mb-sandbox-review/scripts/check-local.py
forge test --match-path test/foundry/PolicyCommitment.t.sol --match-test \
  '^test_OnlyTheOwnerWrites' -vv
forge test --match-test \
  '^test_Ownable2Step_(TransferRequiresAcceptance|AttackerCannotAccept|OnlyOwnerCanPropose)' \
  -vv
```

Expected: six local `PASS` lines; version and hash remain unchanged after rejected writes; ownership changes only after the proposed owner accepts. The Foundry tests are `test_OnlyTheOwnerWrites`, `test_OnlyTheOwnerWritesToEveryRegister` and the three named `test_Ownable2Step_...` tests. Their multisig-named fixture uses dummy addresses; it is not a real multisignature wallet.

Sample audit record

This is the complete JSON fixture. The registry check at block `0x4` returned version 3. A separate GUSD transfer was mined at block `0x5`. These are separate operations; the sample does not show that the transfer enforced the rule. See the record explanation.


```
{
  "eth_chainId": "0x7a69",
  "eth_call": {
    "request": {
      "jsonrpc": "2.0",
      "id": 1,
      "method": "eth_call",
      "params": [
        {
          "from": "0x70997970C51812dc3A010C7d01b50e0d17dc79C8",
          "to": "0x5FbDB2315678afecb367f032d93F642f64180aa3",
          "data":
 "0xdc65a7b5000000000000000000000003c44cdddb6a900fa2b585dd299e03d12fa4293bc000000000000000000000000e7f1725e7734ce288f8367e1bb143e90bb3f0512"
        },
        "0x4"
      ]
    },
    "response": {
      "jsonrpc": "2.0",
      "id": 1,
      "result": "0x0000000000000000000000000000000000000000000000000000000000000003"
    }
  },
  "checkSubscriptionCompliance": {
    "subscriber": "0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC",
    "subscriptionToken": "0xe7f1725E7734CE288F8367e1Bb143E90bb3F0512",
    "policyVersion": "3"
  },
  "PolicyUpdated": {
```

```
    "version": "3",
    "policyHash": "0x9ed40a743048349cd7d7373b555a582ef208f2b91788b460fa34c66e7fa4fc53"
  },
  "Transfer": {
    "from": "0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266",
    "to": "0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC",
    "value": "10000000000000000000"
  },
  "eth_getTransactionReceipt": {
    "transactionHash": "0x412dd06a728b5cf6c46e4e18cc5d9130136e55610c07a16ae1494b0bbcf6af76",
    "blockNumber": "0x5",
    "status": "0x1",
    "from": "0xf39fd6e51aad88f6f4ce6ab8827279cfffb92266",
    "to": "0xe7f1725e7734ce288f8367e1bb143e90bb3f0512",
    "logs": [
      {
        "address": "0xe7f1725e7734ce288f8367e1bb143e90bb3f0512",
        "topics": [
          "0xddf252ad1be2c89b69c2b068fc378daa952ba7f163c4a11628f55a4df523b3ef",
          "0x000000000000000000000000f39fd6e51aad88f6f4ce6ab8827279cfffb92266",
          "0x0000000000000000000000003c44cdddb6a900fa2b585dd299e03d12fa4293bc"
        ],
        "data": "0x0000000000000000000000000000000000000000000000056bc75e2d63100000",
        "blockHash": "0xafe1e69124f1a37fd915d7585dce37a4e8979e32b0b05bb7232829eed16bf613",
        "blockNumber": "0x5",
        "blockTimestamp": "0x6af21ca4",
        "transactionHash": "0x412dd06a728b5cf6c46e4e18cc5d9130136e55610c07a16ae1494b0bbcf6af76",
        "transactionIndex": "0x0",
        "logIndex": "0x0",
        "removed": false
      }
    ]
  },
  "eth_getBlockByNumber": {
    "hash": "0xafe1e69124f1a37fd915d7585dce37a4e8979e32b0b05bb7232829eed16bf613",
    "number": "0x5",
    "timestamp": "0x6af21ca4"
  }
}
```


Limits of this review

The registry judges asset addresses and calling-contract status only. It ignores the subscriber and does not inspect amount, transfer parties or instrument terms. No sukuk token or manager is included. Direct and delegated transfers through the plain GUSD token bypass the registry.

The HTTP mock reads a JSON file. It has no registry adapter or real authentication, and trusts caller-supplied identifiers. It has no transfer executor. A production HTTP outage policy is not implemented. The mock's locked dependencies report one critical, one high and one moderate npm audit issue; the report does not prove exploitability.

The audit example combines separate registry reads, policy and token events, and node responses. It does not bind the check to the transfer. The registry stores a change hash, not a ruling text or identifier; a Majlis registry write is not implemented.

The repository documents no completed independent security audit, says no contract has been deployed to mainnet, and says nothing ships to mainnet before the audit. This review does not establish regulator approval, Shariah certification or a hosted service. The Sepolia observation is dated. No testnet write was sent.

The full evidence and source references cover the exact scope.

What the production version will add

Timing to be agreed. Each item below describes a gap found in this source review.

Transfer enforcement. Today, GUSD never calls the registry. Build a sukuk token or manager that checks the registry inside every balance-changing route: direct, delegated, custody and administrative. Test each route with approved and refused cases. The reviewer must confirm the token and transfer routes.

One-transaction record. Today, the check and transfer are separate. Build a transaction that checks and transfers atomically, and emits the transfer, returned policy version and policy hash. Test that refusal moves no balance and that event fields match the checked rule. Agree the record fields and retention format.

Investor eligibility. Today, `subscriber` is ignored; the registry judges assets and callers only. Add investor and transfer rules for eligibility, amounts and parties as required. Test each field and refusal. The reviewer must define eligibility, authoritative data and privacy constraints.

Registry-backed HTTP. Today, the mock reads a JSON file, has no real authentication and trusts identifiers. Build an authenticated registry adapter, validate addresses, amounts and request context, and resolve the one critical, one high and one moderate npm audit findings. Test invalid requests, auth, registry disagreement and the updated dependencies. Agree the authentication scheme and hosted boundary.

Outages. Today, paused registry checks revert and the example HTTP client exits on request failure without transferring. No production timeout, retry or stale-decision rule exists. Specify bounded timeouts/retries and refuse unavailable or stale decisions. Test timeouts, RPC/API failure and recovery. The reviewer must approve operating limits.

Governance. `docs/KNOWN-ISSUES.md` says ownership remains with the deployer; the engines use single-step `Ownable`, and pausing them waits for the timelock path with no separate pauser. Before handover, add two-step engine ownership and a separate emergency pauser; then configure and test the timelock handover and pause sequence. Agree owners, delay, signers and pauser authority.

Ruling link. Today, the registry records a change hash but no ruling text or identifier; a Majlis registry write is absent. Record a ruling identifier and document digest against each version and emit them with the change. Test that a version resolves to its approved ruling. Supply the authoritative identifier and retention rules.

Audit and mainnet. No independent security audit is documented as complete. Commission and resolve one before any mainnet deployment; the repository says nothing ships to mainnet before the audit. Test the release build and deployment procedure against reviewed source. Approve the audit scope and deployment decision.

Hosted sandbox and shared testnet. No hosted service or keys were verified; the dated registry had no approved asset. Provide a restricted hosted sandbox and a shared testnet exercise only after confirming code and addresses. Test access and write restrictions. Provide endpoint, authentication, RPC, funded non-production accounts and an authorized operator.